

▼ **Práctica 3.1: Generación de Chunks e Indexación Vectorial (RAG)**

Modelo: Qwen/Qwen2.5-1.5B-Instruct

En esta práctica prepararemos el 'cerebro externo' para nuestro modelo de IA. Utilizaremos **LangChain** para procesar documentos y **FAISS** para crear una base de datos vectorial.

```
!pip install pypdf sentence-transformers faiss-cpu transformers huggingface_hub
```

Mostrar salida oculta

```
from huggingface_hub import login
token = "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"
login(token=token)
```

▼ **Importaciones y Configuración del Tokenizador**

```
import os
import json
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
import faiss
from transformers import AutoTokenizer

# Configuración del modelo y almacenamiento
model_name = "Qwen/Qwen2.5-1.5B-Instruct"
STORE_DIR = "/content/drive/MyDrive/rag_store"
os.makedirs(STORE_DIR, exist_ok=True)

# Cargamos el tokenizador oficial para medir los fragmentos
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)

# Parámetros de fragmentación (en tokens)
CHUNK_SIZE = 512
CHUNK_OVERLAP = 50
```

Mostrar salida oculta

▼ **Función de fragmentación (Basada en tokens)**

```
def chunk_text_by_tokens(text, size, overlap):
    # Convertimos el texto a IDs de tokens
    tokens = tokenizer.encode(text, add_special_tokens=False)
    chunks = []

    start = 0
    while start < len(tokens):
        end = start + size
        chunk_tokens = tokens[start:end]
        # Convertimos de nuevo los tokens a texto legible
        chunks.append(tokenizer.decode(chunk_tokens))
        start += size - overlap
    return chunks
```

▼ **Montar carpeta de drive**

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

▼ **Procesamiento de la carpeta de documentos**

```
PDF_DIR = "/content/drive/MyDrive/PDFS_MIA"
documents = []
metadatas = []

if not os.path.exists(PDF_DIR):
    print(f"Error: La carpeta {PDF_DIR} no existe.")
else:
    for pdf_file in os.listdir(PDF_DIR):
        if not pdf_file.lower().endswith(".pdf"):
            continue

        print(f"Procesando: {pdf_file}...")
        reader = PdfReader(os.path.join(PDF_DIR, pdf_file))

        for page_num, page in enumerate(reader.pages):
            text = page.extract_text()
            if not text or len(text.strip()) < 50:
                continue

            # Usamos la nueva función basada en tokens de Qwen
            chunks = chunk_text_by_tokens(text, CHUNK_SIZE, CHUNK_OVERLAP)

            for i, chunk in enumerate(chunks):
                documents.append(chunk)
                metadatas.append({
```

```

        "source": pdf_file,
        "page": page_num + 1,
        "chunk_id": i
    })

print(f"\nTotal chunks generados: {len(documents)}")

Procesando: 1 Introduccion.pdf...
Procesando: 2 Historia.pdf...
Procesando: 3 Implicaciones éticas y sociales.pdf...
Procesando: 4 Modelos, Aprendizaje Automatico y Profundo.pdf...
Procesando: 5 Conjuntos de datos.pdf...
Procesando: 6 Modelos para Machine Learning.pdf...
Procesando: 8 Almacenamiento de los datasets.pdf...
Procesando: 9 ML con Python.pdf...
Procesando: 10 Carga de Datasets.pdf...
Procesando: 11 Consulta de datos con Pandas.pdf...
Procesando: 12 Selección de columnas.pdf...
Procesando: 13 Análisis y adecuación de un dataset.pdf...
Procesando: 13-b Revisión del Target.pdf...
Procesando: 14 Gráficos básicos.pdf...

Total chunks generados: 138

```

- ✓ Generación de Embeddings e Índice FAISS

```
# Generación de Embeddings
embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
embeddings = embedder.encode(documents, convert_to_numpy=True, show_progress_bar=True)

# Creación del Índice FAISS
dim = embeddings.shape[1]
index = faiss.IndexFlatL2(dim)
index.add(embeddings)

# Guardar índice físico
faiss.write_index(index, os.path.join(STORE_DIR, "index.faiss"))

# Guardar textos y metadatos en JSON para recuperarlos después
with open(os.path.join(STORE_DIR, "chunks.json"), "w", encoding="utf-8") as f:
    json.dump(
        [{"text": d, "meta": m} for d, m in zip(documents, metadatas)],
        f,
        ensure_ascii=False,
        indent=2
    )

print("Ingesta completada. Los archivos están en la carpeta 'rag_store'.")
```

```
Loading weights: 100% ██████████ 103/103 [00:00<00:00, 530.31it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | 
-----+-----+--
embeddings.position_ids | UNEXPECTED | 

Notes:
- UNEXPECTED : can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Batches: 100% ██████████ 5/5 [00:00<00:00, 8.19it/s]
Ingesta completada. Los archivos están en la carpeta 'rag_store'.
```